networking details (Figure 6), and this is where the game becomes a bit tricky—which is exactly why we've already set up networking first. You can either have the virtual system to use a virtual network (the *default* interface we discussed earlier), or a shared physical device (your Ethernet card). The latter should appear as "eth0 (Bridge switch)" if you have followed the initial networking set-up steps. I selected this option and moved *Forward*.

This is where we need to allocate the memory and CPU information (Figure 7). I hope you have sufficient memory, as this is the most important aspect of successfully running virtual machines. So, I'd recommend reading the information provided in this screen very carefully. Out of the 2GB memory I have in my system, I allocated 1024 MB as the 'VM Max Memory' and 512 MB as 'VM Startup Memory'. Also, since my CPU is an AMD dual core, I've two logical CPUs; hence, I have defined the number of virtual CPUs for the guest OS as two.

Moving *Forward* will display the summary screen with the different parameters we've set in the configuration steps. Hitting the *Finish* button here boots the install/Live media. This opens a new window where you can see your guest OS boot (Figure 8). You know what to do next, don't you? Use it as you would use any other OS.

This, in fact, is an excellent way to quickly test beta-quality releases like the openSUSE11.1 RC release I am using here. After a successful boot, it picks up an IP supplied by an Airtel broadband router, just like my host OS, and appears to be just another machine in the LAN. In Figure 9, notice that I'm accessing the home directory of the guest OS from my host OS using *ssh* over Nautilus. Note that I'm also accessing the KDE Quick Start Guide from within the guest (where it's actually located), as well as the host.

Similarly, accessing websites from the guest OS is also as simple as launching Firefox and keying in the URL. The guest OS will send the request directly to my router and retrieve the data.

Now, installing this Live CD as a guest OS is also just like you'd install your regular (host) OS. The only difference is, perhaps, the partitioning. Since I exported a file image during the configuration steps for storage space, it'll appear as a blank (raw) hard disk to the guest OS. I accepted the default partition set-up suggested by openSUSE—approximately 500 MB of swap and the rest as root. To tell you the truth, I accepted all the defaults during the installation process to quickly get done with it. Figure 10 shows the guest OS after installation. Installation is not really necessary but... hey, it's possible, so why not?

Of course, there are a few other tweaks possible even after installation. Like what if you would like to increase the maximum memory (RAM) allocation for the guest OS on-the-fly? Click on the *Hardware* tab on the Virtual Machine window. Select the *Memory* parameter from the left pane, and there you have it (Figure 11).

Other hardware changes are also possible, like start-up memory for the VM, the number of virtual CPUs, etc, and can be done offline—that is, after switching off the guest

Figure 10: Guest OS after installation

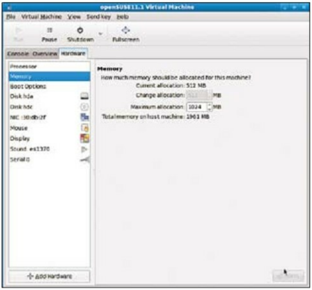

Figure 11: Change max memory available to the guest OS on-the-fly

OS. Go through all the hardware parameters available in the *Hardware* tab to get yourself familiar with it. Clicking the *Overview* tab, on the other hand, gives you information about the CPU and physical RAM that the guest OS is currently using. Pretty simple, isn't it?

Closing up

Finally, my business is done here! Obviously, there's a lot, lot more that KVM (and even the GUI wrapper called Virt-Manager) is capable of; I've only managed to touch base with the easily-accessible upper crust. This Virt-Manager plus KVM is a new thing for me, so I wanted to share my explorations with all of you. Catch you later when I have something else to rant about. END

By: Atanu Datta

He likes to head bang and play air guitar in his spare time. Oh, and he's also a part of the LFY Bureau.